به نام خدا

دانشکده فنی و مهندسی

درس: طراحی سیستم‌های دیجیتال(FPGA و ASIC)

پروژه پایانی

استاد درس : خانم دکتر امینی

دستیار آموزشی: خانم ملیکا مرسلی

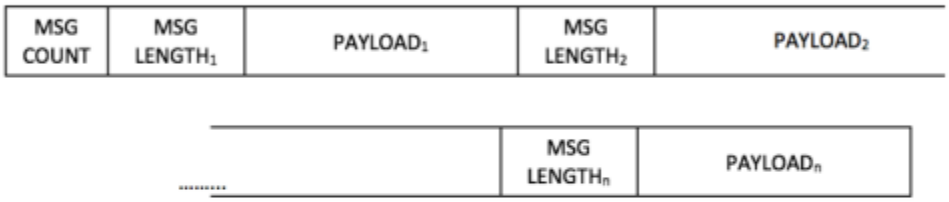
نام و نام خانوادگی:

ماندانا مرادی - 9915302007

1. صورت سوال:

هدف از این پروژه طراحی یک استخراج کننده پیام می باشد. این سیستم payload های مورد نظر را از داده ورودی که شامل بسته هایی به صورت شکل 1 می باشد استخراج می کند. داده ورودی به صورت جریانی از 64 بیت داده در هر سیکل کلاک می باشد.

توضیح هر بسته که در ورودی دریافت می شود به صورت جدول 1 می باشد.

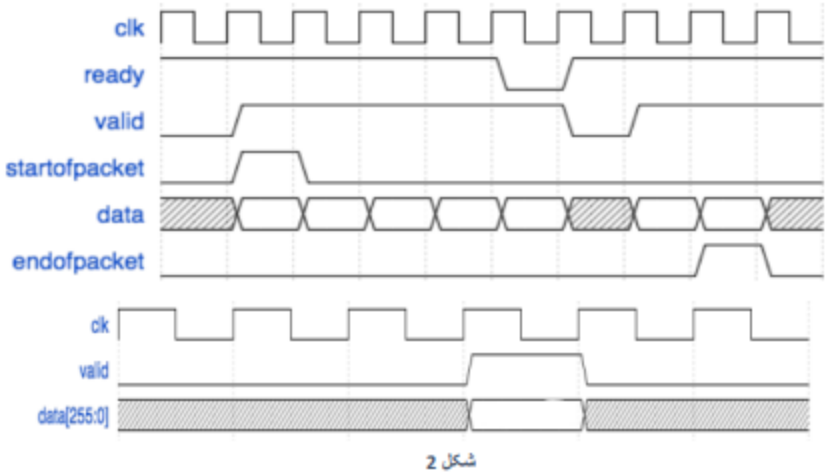


شکل 1

جدول 2

نام بسته	طول	توضیح
MSG COUNT	1 بایت	تعداد payload ها را مشخص می کند.
MSG LENGTH	1 بایت	طول payload بعدی مشخص می کند.
PAYLOAD	متغیر	پیامی که باید استخراج شود.

سیگنال ها و ست آپ ورودی و خروجی به صورت شکل 2 است و توضیحات هر یک در جدول 2 شرح داده شده است:



شکل 2

جدول 2

نام سیگنال	جهت	بیت	توضیح
CLK	input	1	کلاک سیستم
reset	input	1	ریست active low
ready	output	1	سیستم آمده دریافت داده است. (تاخیر خواندن یک سیکل می باشد)
Valid_in	input	1	زمانی که ورودی معتبر است یک می باشد در غیر این صورت صفر است.
startofpacket	input	1	برای اولین سیکل دریافت داده یک می باشد در غیر این صورت صفر است.
Data_in	input	64	داده ورودی (از LSB شروع می شود)
endofpacket	input	1	برای آخرین سیکل دریافت داده یک می باشد در غیر این صورت صفر است.
Valid_out	output	1	زمانی که خروجی معتبر است یک می باشد در غیر این صورت صفر است.
Data_out	output	128	داده خروجی،payload استحراج شده.

فرض کنید حداقل طول payload 6 بایت و حداکثر طول آن 14 بایت باشد.

با توجه به سیگنال های ورودی و خروجی و نوع هماهنگی بین آن ها در شکل 2 و جدول 2 ، یک استخراج کننده پیام که جریان داده آن به صورت شکل 1 و جدول 1 می باشد پیاده سازی کنید.

- ماشین حالت سیستم فوق باید رسم گردد.
- سیستم باید با دو PROCESS پیاده سازی شود.
- شبیه سازی رفتاری برای سیستم فوق با مثال صفحه بعد باید انجام گیرد.
- حداکثر فرکانس کاری مدار و گزارش های مربوط به TIMING باید شرح داده شود.

نمونه مثال:

توجه: لطفاً در نظر بگیرید که بایت MSB آخرین بایت و بایت LSB اولین بایتی است که دریافت شده است.

نمونه سمپل ورودی

Data_in [63:0] (hex)	startofpacket	endofpacket
62626261616115e6	1	0
6968686783938386	0	0
5a5a6464d4e4d4e8	0	0
XXXXX59097b7b7ba	0	1

نمونه سمپل خروجی

Data_out (hex)	Valid_out
62626261616115	1
393838	1
69686867	1
4d4e4d4e	1
5a5a64	1
59097b7b7b	1

2. حل مسئله:

حل دستی:

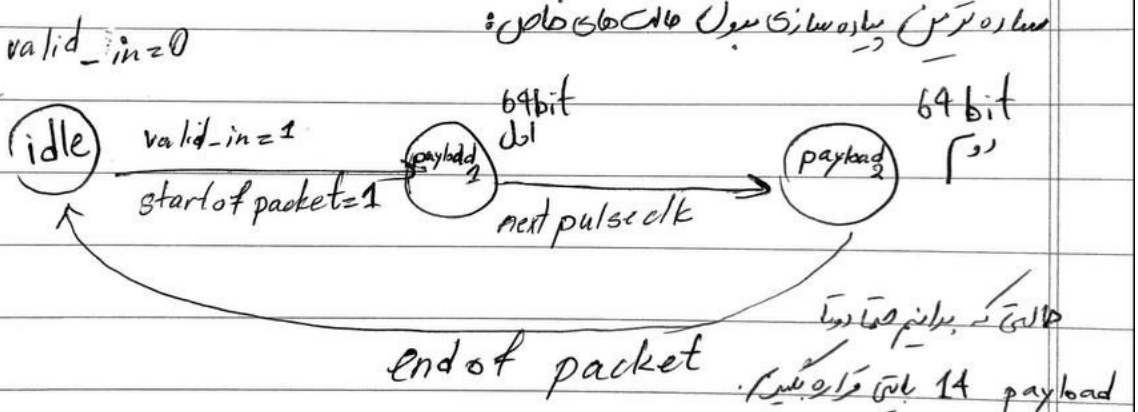
با توجه به اینکه حروف ما 128bit است و اینکه payload ها مینیمم 6 بایت و ماکزیمم 14 بایت هستند:

به صورت مینیمم می‌توانیم دوتا payload 14 بایتی داشته باشیم، ماکزیمم 6 payload بایتی داشته باشیم.

min: $14 \times 4 = 56$, $56 \times 2 = 112$, $128 - 112 = 16$

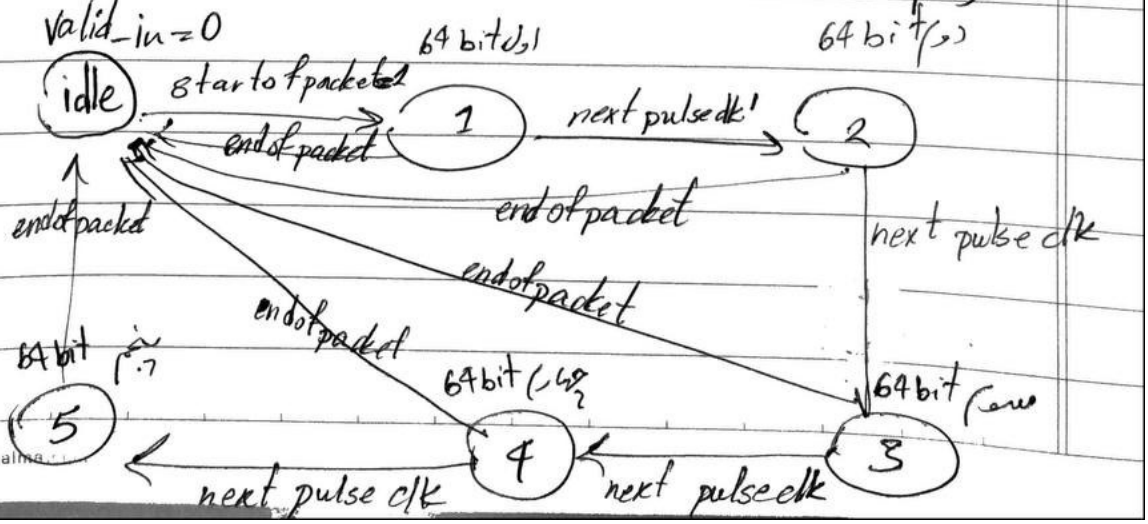
max: $6 \times 4 = 24$, $24 \times 5 = 120$, $128 - 120 = 8$

ساده‌ترین پیاده‌سازی بدون حالت‌های خاص:



حالتی که برایم حیاتی است 14 payload بایتی فرا برسد.

ماکزیمم تعداد payload ها:



www.nahal-alma

کد زیر را می زنیم.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;
-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;
entity Final_Proj is
     Port (
     clk : in std_logic;
     reset : in std_logic;
     ready : in std_logic;
     Valid_in : in std_logic;
     startofpacket : in std_logic;
     Data_in : in std_logic_vector(63 downto 0);
     endofpacket : in std_logic;
     Valid_out : out std_logic;
     Data_out : out std_logic_vector(127 downto 0)
      );
end Final_Proj;
architecture Behavioral of Final_Proj is
    TYPE state is (idle, one, two);
    SIGNAL pr_state,nx_state :state;
    signal MSG_COUNT : std_logic_vector(3 downto 0) :=(others=>'0');
    signal MSG_LENGTH1 :std_logic_vector(3 downto 0) :=(others=>'0');
    signal payload1 : std_logic_vector(55 downto 0):=(others=>'0');
    signal MSG_LENGTH2 :std_logic_vector(3 downto 0) :=(others=>'0');
    signal payload2 : std_logic_vector(55 downto 0):=(others=>'0');
        signal x : std_logic_vector(15 downto 0) := (others=>'0');
```

```
begin
    -------------Lower section------------
    process(reset,clk)
    begin
        if(reset='1')then
            pr_state<=idle;
        elsif(rising_edge(clk))then
            pr_state<=nx_state;
        end if;
    end process;
    ------------Upper section------------
    process(pr_state,Valid_in,Data_in)
    begin
        if(Valid_in='1')then
            case pr_state is
                 when idle =>
                    if (startofpacket = '1')then
                        nx_state<=one;
                    else
                         nx_state<=idle;
                    end if;
                 when one =>
                     payload1<=Data_in(63 downto 8);
                     MSG_LENGTH1<=Data_in(7 downto 4);
                     MSG_COUNT<=Data_in(3 downto 0);
                     nx_state<=two;
                 when two =>
                   if(endofpacket = '1')then
                          nx_state<=idle;
                          Valid_out <='1';
                   else
                          payload2<=Data_in(59 downto 4);
```

```
                        MSG_LENGTH2<=Data_in(3 downto 0);
                        nx_state<=idle;
                        Valid_out <='0';
                    end if;
          end case;
        end if;
    end process;
        Data_out<=(x & payload1 & payload2);
end Behavioral;
```

توضیح کد:

در **entity** ابتدا ورودی ها و خروجی ها رو تعریف می کنیم.

```
TYPE state is (idle, one, two);
 SIGNAL pr_state,nx_state :state;
```

برای پیاده سازی با **fsm** ابتدا داده **enumerated** تعریف می کنیم. و سیگنال حالت فعلی و حالت بعدی تعریف می کنیم.

سپس سیگنال های میانی را تعریف می کنیم.

```
process(reset,clk)
    begin
        if(reset='1')then
            pr_state<=idle;
        elsif(rising_edge(clk))then
            pr_state<=nx_state;
        end if;
    end process;
```

یک پروسس می نویسیم که در لیست حساسیت آن ریست و کلاک را قرار می دهیم.

در صورت یک شدن ریست حالت کنونی را روی حالت اولیه قرار می گیرد. در غیر این صورت با آمدن پالس کلاک حالت بعدی در حالت کنونی قرار می گیرد.

```
process(pr_state,Valid_in,Data_in)
    begin
```

```
    if(Valid_in='1')then
        case pr_state is
             when idle =>
                if (startofpacket = '1')then
                    nx_state<=one;
                else
                     nx_state<=idle;
                end if;
             when one =>
                 payload1<=Data_in(63 downto 8);
                 MSG_LENGTH1<=Data_in(7 downto 4);
                 MSG_COUNT<=Data_in(3 downto 0);
                 nx_state<=two;
                 Valid_out<='0';

             when two =>
               if(endofpacket = '1')then
                      nx_state<=idle;
                      Valid_out <='1';
               else
                      payload2<=Data_in(59 downto 4);
                      MSG_LENGTH2<=Data_in(3 downto 0);
                      nx_state<=idle;
                      Valid_out <='0';
               end if;
      end case;
    end if;
end process;
```

در لیست حساسیت پروسس دوم دیتای ورودی، Valid_in ، و pr_state را قرار می دهیم.

در صورت یک بودن Valid_in و معتبر بودن داده وارد یک ساختار Case می شویم. در صورت آمدن Startofpacket وارد حالت بعدی یعنی خواندن پیام شماره ی یک می شویم.

در حالت شماره 1 :

```
payload1<=Data_in(63 downto 8);
                    MSG_LENGTH1<=Data_in(7 downto 4);
                    MSG_COUNT<=Data_in(3 downto 0);
                    nx_state<=two;
```

بیت شماره ی 0 تا 3 از اولین رشته بیت 64 تایی MSG_Count است که به آن ارجاع می دهیم.

بیت شماره ی 4 تا 7 طول پیام اوی را مشخص می کند که به سیگنال مربوطه ارجاع می دهیم.

بیت شماره ی 8 تا 63 پیام اول است که به سیگنال Payload1 ارجاع می دهیم.

و در آخر حالت بعدی را در سیگنال nx_state قرار می دهیم.

در حالت شماره 2:

```
if(endofpacket = '1')then
                        nx_state<=idle;
                        Valid_out <='1';
                else
                        payload2<=Data_in(59 downto 4);
                        MSG_LENGTH2<=Data_in(3 downto 0);
                        nx_state<=idle;
                        Valid_out <='1';
                end if;
```

در صورت آمدن endofpacket حالت بعدی حالت اولیه خواهد بود و خروجی معتبر می شود. در غیر این صورت پیام شماره 2 را دریافت می کنیم. طول پیام دوم و خود پیام دوم را به سیگنال های مربوطه ارجاع می دهیم. حالت بعدی را حالت اولیه قرار می دهیم و در این صورت نیز خروجی معتبر می شود.

و در آخر خارج از پروسس ها و در محیط concurrent خروجی را در data_out می ریزیم.

کد تست بنچ را به صورت زیر می زنیم:

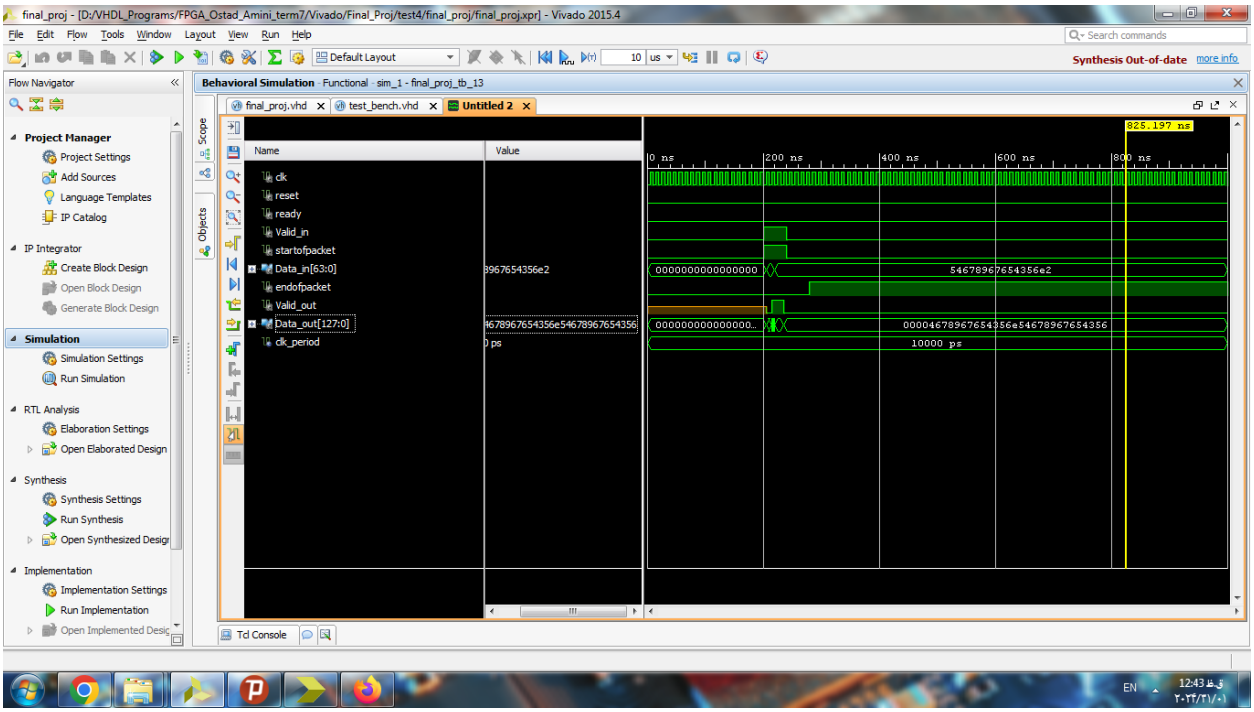
```
LIBRARY ieee;
```

```
USE ieee.std_logic_1164.ALL;
-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;
ENTITY final_proj_tb_13 IS
END final_proj_tb_13;
ARCHITECTURE behavior OF final_proj_tb_13 IS
    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT Final_Proj
    PORT(
         clk : IN  std_logic;
         reset : IN  std_logic;
         ready : IN  std_logic;
         Valid_in : IN  std_logic;
         startofpacket : IN  std_logic;
         Data_in : IN  std_logic_vector(63 downto 0);
         endofpacket : IN  std_logic;
         Valid_out : OUT  std_logic;
         Data_out : OUT  std_logic_vector(127 downto 0)
        );
    END COMPONENT;
   --Inputs
   signal clk : std_logic := '0';
   signal reset : std_logic := '0';
   signal ready : std_logic := '0';
   signal Valid_in : std_logic := '0';
   signal startofpacket : std_logic := '0';
   signal Data_in : std_logic_vector(63 downto 0) := (others => '0');
   signal endofpacket : std_logic := '0';
       --Outputs
   signal Valid_out : std_logic;
   signal Data_out : std_logic_vector(127 downto 0);
```

```
   -- Clock period definitions
   constant clk_period : time := 10 ns;
BEGIN
       -- Instantiate the Unit Under Test (UUT)
   uut: Final_Proj PORT MAP (
          clk => clk,
          reset => reset,
          ready => ready,
          Valid_in => Valid_in,
          startofpacket => startofpacket,
          Data_in => Data_in,
          endofpacket => endofpacket,
          Valid_out => Valid_out,
          Data_out => Data_out
        );
   -- Clock process definitions
   clk_process :process
   begin
              clk <= '0';
              wait for clk_period/2;
              clk <= '1';
              wait for clk_period/2;
   end process;
   -- Stimulus process
   stim_proc: process
   begin
      -- hold reset state for 100 ns.
      wait for 100 ns;
      wait for clk_period*10;
      -- insert stimulus here
      wait;
   end process;
```

```
END;
```

3. نتیجه:


final_proj - [D:/VHDL_Programs/FPGA_Ostad_Amini_term7/Vivado/Final_Proj/test4/final_proj/final_proj.xpr] - Vivado 2015.4
File Edit Flow Tools Window Layout View Run Help
Synthesis Out-of-date more info
Flow Navigator
Project Manager
Project Settings
Add Sources
Language Templates
IP Catalog
IP Integrator
Create Block Design
Open Block Design
Generate Block Design
Simulation
Simulation Settings
Run Simulation
RTL Analysis
Elaboration Settings
Open Elaborated Design
Synthesis
Synthesis Settings
Run Synthesis
Implementation
Implementation Settings
Run Implementation
Behavioral Simulation - Functional - sim_1 - final_proj_tb_13
final_proj.vhd
test_bench.vhd
Untitled 2
Name
Value
clk
reset
ready
Valid_in
startofpacket
Data_in[63:0]
endofpacket
Valid_out
Data_out[127:0]
clk_period
825.197 ns
0 ns
200 ns
400 ns
600 ns
800 ns
10000 ps
Tcl Console